

ASV Simulator Design Document (Updated)

1. Overview

This document describes the design of a Python-based simulator for an Autonomous Surface Vehicle (ASV) tasked with navigating a pre-qualification course. The course involves passing through a gate, maneuvering around a marker pole, and returning through the gate. The simulator provides a 2D top-down map view and **two** simulated 3D first-person camera views (**front-facing** and **right-side/starboard**). It models an ASV operating on the surface ($Z=0$) using a differential drive (two thrusters). The AI employs a modular Task/Subtask architecture for mission execution and relies on simulated computer vision (OpenCV blob detection) for navigation using both camera views.

2. Key Program Concepts

- **Task/Subtask Architecture:** The core AI logic is structured hierarchically:
 - **Tasks** (`ai.tasks.task_base.Task`): Represent high-level mission goals (e.g., `GateTask`, `side_orbit_task` defined in `main.py`). Each task manages a sequence of subtasks. Tasks receive **two updated Vision objects** (`front_vision`, `side_vision`) in their `execute` method.
 - **Subtasks** (`ai.tasks.subtask_base.Subtask`): Represent reusable, atomic actions (e.g., `TurnToHeading`, `DriveStraight`, `AlignToObjectX`, **`ApproachObjectByWidth`**, **`AlignUsingSideCamera`**, **`OrbitUsingSideCamera`**, **`DriveThroughGate`**). They are executed sequentially by the parent Task. Subtasks receive the updated `front_vision` and `side_vision` objects in their `execute` (and `on_enter/on_exit`) methods. They call the interpretation methods of the relevant Vision object (e.g., `front_vision.is_pole_visible()`, `side_vision.get_pole_apparent_width()`) to get the data needed for their actions.
- **Context Dictionary:** A shared dictionary (`Task.context`) is used for indirect communication between subtasks within a Task. For instance, an alignment subtask stores the final heading, which a subsequent turning subtask can retrieve.
- **Physics Model** (`simulator.py:applyPhysics`): Simulates a simplified 2D rigid body model for the ASV.
 - **Differential Drive:** Calculates surge (forward/backward) force and yaw (turning) torque based on independent port and starboard thruster commands.

- **Drag:** Implements distinct quadratic drag coefficients for surge (surgeDragCoeff), sway (swayDragCoeff, significantly increased for better realism), plus angular drag (angularDragCoeff).
- **Inertia:** Includes mass (subMass) and rotational inertia (subInertia).
- **Surface Constraint:** The boat's Z-position and pitch angle are locked to zero.
- **Simulated Vision** (Vision Class & ai/vision.py):
 - The core vision processing is encapsulated within the Vision class defined in data_structures.py.
 - **Initialization:** Two instances of the Vision class are created within the Submarine AI's constructor: front_vision and side_vision. Each receives an image_provider function (accessing SensorSuite.camera_image or SensorSuite.side_camera_image) and configuration parameters.
 - **Processing Trigger:** At the beginning of each Submarine.update cycle, **both** front_vision.update() and side_vision.update() methods are called. These methods fetch the current camera image, run blob detection (find_blobs_hsv), store the raw blob lists, and clear cached interpretations.
 - **Interpretation:** The Vision class provides methods (e.g., is_pole_visible(), get_gate_center_x(), get_pole_apparent_height() for the front camera; **is_pole_visible_side()**, **get_pole_center_x_side()**, **get_pole_apparent_width()** for the side camera) that interpret the internally stored blob lists. The get_best_pole_side method **does not use a strict shape filter**, relying instead on the largest detected blob being the target. These methods often cache results.
 - **Blob Detection** (ai/vision.py): The find_blobs_hsv function remains the low-level tool using OpenCV (cv2) for blob finding based on HSV color ranges and minimum size.
- **Configuration** (config.py): Centralizes constants, tuning parameters (PID gains, drag coefficients, AI thresholds), color definitions, and course layout.

3. Program Structure

- **main.py:**
 - Main entry point.

- Defines the mission plan as an ordered list of Task objects (e.g., GateTask, a custom side_orbit_task). Sets task-specific parameters like search_direction.
- Instantiates the Submarine (AI controller) and the SubmarineSimulator.
- Starts the simulation loop (sim.run()).
- **simulator.py:**
 - Contains the SubmarineSimulator class.
 - Initializes Pygame, creates the display window, and **two camera surfaces** (cameraSurface, sideCameraSurface).
 - Loads assets (e.g., camera background image).
 - Manages the main simulation loop (run method):
 - Handles user input (handleInput).
 - Updates the AI (submarineAI.update).
 - Applies physics (applyPhysics).
 - Generates **both** 3D camera views (generateCameraView, generateSideCameraView using project3D and project3DSide respectively). project3DSide includes a physical offset for the camera position.
 - Renders the 2D map, UI elements, and **both camera views** (render, _renderUi). Includes debug drawing (e.g., detected blob outlines) on both camera surfaces.
 - Defines physics parameters (mass, inertia, drag coefficients - including increased swayDragCoeff).
 - Creates the course elements (resetSimulation).
- **config.py:**
 - Defines color constants, HSV ranges.
 - Contains dataclasses (SimulationConfig, PrequalConfig) holding world dimensions, camera FOV, boat dimensions, course layout, physics constants, and AI tuning gains.
- **world.py:**

- Defines dataclasses for physical objects (PrequalGate, PrequalMarker).
- Defines SubmarinePhysicsState dataclass.
- **data_structures.py:**
 - Defines simple dataclasses: ThrusterCommands, SensorSuite (now includes side_camera_image), MPU6050Readings.
 - Defines the Vision class, encapsulating blob detection and interpretation methods for **both front and side views**.
- **utils.py:** Contains helper functions (e.g., angle_diff).
- **ai/:**
 - **submarine.py:**
 - Contains the main Submarine AI class.
 - `__init__`: Stores the mission plan, initializes AI gains. Creates and stores **two Vision object instances** (front_vision, side_vision).
 - `update`: Calls `update()` on both vision objects. Executes the current Task, passing **both vision objects**. Handles task transitions. Returns both vision objects for debug drawing.
 - `reset`: Resets mission state.
 - Low-level control methods (e.g., `_mix_and_normalize_commands`, `get_heading_commands`).
 - **vision.py:** Contains `find_blobs_hsv` using OpenCV.
 - **tasks/:**
 - **task_base.py:** Defines Task base class and TaskStatus. Its `execute` method runs the current subtask, passing **both vision objects**.
 - **subtask_base.py:** Defines Subtask base class and SubtaskStatus. Method signatures expect **both vision objects**.
 - **common_subtasks.py:** Implements reusable Subtasks. Vision-based subtasks now use methods of either `front_vision` or `side_vision`. Includes **ApproachObjectByWidth**, **AlignUsingSideCamera**, **OrbitUsingSideCamera**, and

DriveThroughGate. Does **not** contain CreepUntilSideVisible or ApproachAndCenterObject anymore.

- **gate_task.py:** Task for gate navigation using common subtasks, including DriveThroughGate.
- **stabilize_task.py:** Task wrapper around stabilization logic.
- *(Obsolete: ratchet_turn_task.py, marker_turn_task.py, drive_straight_task.py)*

4. Transitioning to Hardware

(This section remains largely the same, emphasizing the need for tuning PID gains, vision thresholds, and subtask parameters like target widths and surge powers.)

- Replace SensorSuite population with real sensor drivers/fusion.
- Retune config.py HSV ranges extensively. Vision class structure remains valid.
- Map ThrusterCommands output to ESC signals.
- Remove applyPhysics.
- Retune all PID gains and AI parameters (e.g., YAW_D_GAIN, orbit gains, target_pole_width_px) on the physical boat.
- Account for real-world latency, noise, calibration needs, power, and environmental factors.

5. Task and Subtask Details

5.1 GateTask - Navigating the Gate

The GateTask finds the gate (red poles), aligns, and drives through.

1. **WaitForTargetVisible(target_type='gate'):** Spins until front_vision.is_gate_visible() is True.
2. **AlignToObjectX(target_x_fraction=0.5, ...):** Uses front_vision.get_gate_center_x() to point at the gate center. Stores heading.
3. **Stabilize(duration=1.0, ...):** Stops residual motion.
4. **DriveThroughGate(surge_power=0.6, ...):** Drives forward while using front_vision.get_gate_center_x() and yaw control to actively steer towards the gate center. Completes when gate is lost.

5. **DriveStraight(duration=4.0, ...):** Ensures boat fully clears the gate.
6. **Stabilize(duration=1.0):** Stops after clearing the gate.

5.2 Side Orbit Task (side_orbit_task in main.py) - Maneuvering Around the Marker

This custom Task uses a sequence of subtasks to navigate around the green marker pole.

1. **WaitForTargetVisible(target_type='pole'):** Finds the green pole using the **front camera**.
2. **ApproachObjectByWidth(target_width_px=35, ...):** Drives towards the pole using the **front camera**, keeping it centered and stopping when its apparent **width** reaches the target value (e.g., 35px). Stores heading.
3. **Stabilize(duration=1.0, ...):** Stops motion after approaching.
4. **AlignUsingSideCamera(search_yaw_power=0.15, ...):** Yaws **right** (search_yaw_power > 0) until the pole is detected in the **side camera**, then uses PD control (align_yaw_gain) on yaw to center the pole (target_x_fraction=0.5) in the **side camera** view. Completes when centered and stable. Stores heading.
5. **OrbitUsingSideCamera(target_pole_width_px=40, ...):**
 - **Goal:** Circle the pole while maintaining distance until the return gate is seen.
 - **Action:** Commands constant forward orbit_surge_power. Uses the **side camera** for control.
 - **Control (Yaw):** Combines multiple terms:
 - P-control (yaw_x_gain, positive) based on side_vision.get_pole_center_x_side() to keep the pole centered.
 - PI-control (yaw_dist_p_gain, negative; yaw_dist_i_gain, negative) based on side_vision.get_pole_apparent_width() error (target_pole_width_px - current_width) to maintain distance. The integral term helps correct steady drift.
 - D-control based on sensors.imu.gyro_z (sub.YAW_D_GAIN) for stability.
 - **Completion:** Runs until front_vision.is_gate_visible() returns True, then immediately stops orbiting.

6. **AlignToObjectX(target_x_fraction=0.5, ...):** After spotting the gate, uses the **front camera**(front_vision.get_gate_center_x()) to align the boat towards the center of the return gate.